R1.01 — Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Initiation au développement et conception algorithmique

Synthèse exécutive : Ce cours pose les bases indispensables de la conception algorithmique rigoureuse en BUT 1 Informatique. Avant d'écrire la moindre ligne de code, l'étudiant doit identifier le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours détaille l'utilisation méthodique des trois modèles canoniques sur structures à accès direct (tableaux, chaînes de caractères) : le parcours complet systématique, le parcours complet conditionné et la recherche de première occurrence. Une attention toute particulière est accordée à la prévention des erreurs de dépassement d'indice (out of bounds) ainsi qu'aux exigences méthodologiques de la recherche (pessimisme algorithmique et double condition d'arrêt obligatoire).

Définitions et Typologie des Mécanismes d'Accès aux Données

L'analyse préalable du mécanisme d'accès constitue l'étape zéro de toute conception algorithmique. Si une structure possède un accès direct (comme un tableau de N éléments ou une chaîne de caractères), tout élément situé à la position 'i' est immédiatement accessible par l'instruction T[i]. À l'inverse, sur une structure à accès séquentiel (tel qu'un fichier texte brut, un flux CSV ou une liste chaînée simple), il est impossible d'accéder directement à une position arbitraire comme T[50] : il est obligatoire d'exécuter 50 fois l'opération 'suivant' à partir du début. Tenter d'appliquer un algorithme d'accès direct (comme une boucle Pour i de 0 à N-1 avec accès T[i]) sur un flux séquentiel constitue une erreur de conception fondamentale qui invalide totalement l'algorithme.

- Structure de données : Collection organisée regroupant des éléments de même type.
- Accès Direct : Capacité d'atteindre instantanément n'importe quel élément via son indice ou sa clé.
- Accès Séquentiel : Contrainte logique ou matérielle imposant de parcourir les éléments un par un depuis le début.
- Accès Dynamique : Structure hybride combinant primitives d'accès séquentiel et direct.

Cas concret enseignant : *Le professeur insiste sur un piège classique en évaluation : utiliser une notation d'accès direct T[i] sur un fichier CSV brut ou un fichier texte. Ces flux étant gérés séquentiellement ligne par ligne par le système, une telle opération est physiquement impossible et entraîne une note éliminatoire sur la question.*

Modèle 1 : Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une collection doit subir un traitement uniforme et inconditionnel (par exemple : initialiser un tableau, afficher tous ses éléments, ou passer tous les caractères d'une chaîne en majuscules). Le nombre d'éléments à traiter étant fixe et déterminé dès le départ (Nombre = Fin - Début + 1), la structure itérative obligatoire est la boucle bornée 'Pour'. Sur un tableau de N éléments indicé de 0 à N-1, la boucle s'écrit 'Pour i allant de 0 à N - 1'. L'erreur classique consiste à faire varier 'i' jusqu'à N, ce qui tente d'accéder à la case N (inexistante) et déclenche une erreur fatale de dépassement d'indice (out of bounds).

- Exhaustivité : 100% des éléments de la zone définie subissent le traitement.
- Structure itérative : Boucle bornée 'Pour' (nombre d'itérations connu à l'avance).
- Gestion des bornes : Pour une collection de N éléments indicée à partir de 0, l'indice varie strictement de 0 à N - 1.

Cas concret enseignant : *Exemple d'illustration du prof : Passer en majuscules tous les caractères d'un mot de passe ou afficher toutes les cases d'un tableau d'entiers. Le traitement est appliqué aveuglément à chaque itération.*

Modèle 2 : Parcours Complet avec Traitement Conditionné

Le parcours complet conditionné conserve l'exploration exhaustive du modèle 1, mais protège l'action de traitement par une condition logique. Il est utilisé pour compter, afficher ou transformer uniquement une sous-population

d'éléments répondant à un critère précis (par exemple : compter les voyelles dans une chaîne, ou afficher les nombres pairs d'un tableau). La condition peut être simple (ex: T[i] > 0), complexe (ex: T[i] > 0 ET T[i] mod 2 = 0), ou impliquer des éléments voisins, sous réserve d'ajuster rigoureusement les bornes pour éviter tout débordement mémoire.

- Inspection complète : Tous les éléments de la zone sont visités de la borne initiale à la borne finale.
- Filtrage logique : Le traitement n'est exécuté que si l'élément courant valide une condition booléenne.
- Combinaison de structures : Imbrication d'une structure alternative 'Si ... Alors' à l'intérieur d'une boucle bornée 'Pour'.

Cas concret enseignant : *L'analogie de la chaîne de montage : Dans une usine de contrôle qualité, chaque pièce défile devant l'inspecteur (parcours complet de la chaîne), mais l'inspecteur n'applique un marquage de rebut (traitement conditionné) que si la pièce présente un défaut spécifique.*

Modèle 3 : Recherche de Première Occurrence

Contrairement aux parcours complets, la recherche de première occurrence s'appuie sur une itération non bornée ('Tant que'), car le nombre de répétitions n'est pas connu à l'avance. Un algorithme de recherche valide repose sur le principe du pessimisme : on suppose au départ que l'élément n'existe pas (trouve <- Faux). Il doit impérativement comporter deux conditions d'arrêt : le succès (l'élément est trouvé) et l'échec (la fin de la structure est atteinte).

- Objectif : Localiser le tout premier élément vérifiant une propriété et interrompre immédiatement l'exploration.
- Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux avant d'entrer dans la boucle.
- Double condition d'arrêt obligatoire : La recherche doit s'arrêter SI l'élément est trouvé OU SI la structure est entièrement parcourue.
- Variante 1 (Rupture immédiate) : S'arrête exactement à l'indice où l'élément a été trouvé.
- Variante 2 (Tant que global) : Regroupe la double condition en tête de boucle ('Tant que i < N et NON trouvé').

Cas concret enseignant : *Devise du prof : 'En informatique, soyez pessimistes ! Partez du principe que ce que vous cherchez n'existe pas, et ne changez votre indicateur que lorsque vous le voyez.' Rédiger une recherche avec une seule condition d'arrêt constitue une erreur majeure sanctionnée en partiel.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, à condition d'utiliser les primitives adaptées pour connaître la taille courante de la structure dynamique, car la boucle 'Pour' nécessite de connaître les bornes de début et de fin.

Question étudiant : Pourquoi présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La variante 1 avec rupture immédiate est idéale lorsque l'on a besoin de réutiliser l'indice exact de l'élément trouvé après la boucle. La variante 2 avec test unique en tête est plus compacte mais nécessite parfois un réajustement de l'indice en raison de l'incrémentation finale.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut adapter strictement les bornes de la boucle 'Pour' (par exemple de 1 à N-1 si on compare T[i] avec T[i-1]) pour ne jamais provoquer de dépassement d'indice aux extrémités.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (ex: trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser une recherche de première occurrence (qui s'arrête au premier trouvé), mais un parcours complet avec traitement conditionnel qui va tester chaque case et enregistrer ou afficher toutes les occurrences.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en utilisant le modèle algorithmique approprié.

Résolution pas à pas :
1. Analyse du problème : Il faut examiner la totalité des éléments d'un tableau (structure à accès direct) de la première à la dernière case. L'affichage (traitement) ne doit s'exécuter que si la valeur est strictement supérieure à 0. Le modèle adapté est le 'Parcours complet avec traitement conditionné'. 2. Détermination des bornes : Le tableau comprend N cases indicées de 0 à N - 1. Borne de début = 0, Borne de fin = N - 1. 3. Choix de la boucle : Une boucle bornée 'Pour i allant de 0 à N - 1'. 4. Condition de filtrage : À chaque itération, tester 'Si T[i] > 0 Alors'. 5. Action : Exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données : T : Tableau d'entiers, N : Entier Variables : i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit un parcours exhaustif sans risque de débordement d'indice (bornes 0 à N-1) et filtre exactement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le modèle 3 : Recherche de première occurrence (avec une boucle Tant que et un arrêt dès que la valeur est trouvée ou la fin du tableau atteinte).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que les tableaux en informatique sont indexés à partir de 0. Un tableau de N éléments possède donc des indices allant de 0 à N - 1. Tenter d'accéder à l'indice N provoque une erreur de dépassement mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt : le succès (l'élément est trouvé) et l'échec (la structure a été entièrement parcourue sans succès).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765